

# OOP Coursework

This coursework covers the first half of the course and while it has a final deadline at the end of the first half of the semester, you will need to make regular commits on gitlab to achieve full marks. Below, we will provide you with an overall idea of the course work and specific tasks for you to finish each week.

## Set-up Instructions

Please, visit [https://gitlab-student.macs.hw.ac.uk/f27sb\\_2023-24/f27sb\\_2023-24\\_coursework/oop\\_coursework](https://gitlab-student.macs.hw.ac.uk/f27sb_2023-24/f27sb_2023-24_coursework/oop_coursework) and create your own fork of this project. Once the fork has been created, import the starter code into Eclipse. Should you struggle with any of this, please have a look at the “How to videos” on the course page on Canvas. If you should still face any problems, please talk to a lab helper during your lab slot.

## The Coursework

For this coursework, you will need to implement a task management system with a simple text-based user interface. The system should support several different types of tasks and should allow the user to add new ones and remove existing ones. Please, pay close attention to the rubric at the end of the document.

The implementation of this system needs to follow the OOP principles that we discuss in the lectures, and it should include comments that clarify how your code addresses principles such as cohesion and coupling. Each week, you will be asked to complete a different part of the system and it is important that you do so because this will guide you to the most desirable solution for this coursework. Moreover, **if your git commit history does not show incremental development of the final solution, you will receive 0 marks for this coursework.**

## Weekly tasks:

Week 1-2 – suggested deadline: end of week 2

- Implement a **Task** class which contains an ID, a name, and a description of the task. The ID and name are mandatory fields of the Task meaning that each instance of Task should make sure that these are specified. The description is optional and can be omitted by the user. Create as many constructors for this class as required. Finally, implement a way to generate a textual representation of this class.
- Implement a **TaskManager** class which uses a collection to store all the instances of Task that the user creates. This class should contain the following methods:
  - `addTask` which adds a new task to the current list of tasks. No two tasks with the same ID should be allowed to be added to the collection of tasks. If a new task with an already existing ID is given as a parameter, this method should return a value that shows that this operation has been unsuccessful and ignore the new task.
  - `removeTask` which uses the ID of a task to find it in the list and remove it. Make sure to return success or failure of this operation because this should only work if a task with the given ID exists in the list of current tasks.
  - `getSize` which returns the current number of tasks in the list.

- a way to get a textual representation of all the tasks in the list.
- Implement a text-based user interface. This should display a welcome message and instructions on how to use the system. Use a `Scanner` object to get the input from the user. The user operations you need to implement are:
  - Show all current tasks
  - Add a new task
  - Remove an existing task
  - Exit the program
- You can find a good example of how to use a `Scanner` for user input here: [https://www.w3schools.com/java/java\\_user\\_input.asp](https://www.w3schools.com/java/java_user_input.asp)
- All implemented classes need to ensure high cohesion and low coupling. Make sure to comment on how you achieved this in the code.

#### Week 3 – suggested deadline: end of week 3

- Implement a unit test for your `TaskManager` class. This should test the `addTask` and `removeTask` methods. Make sure to cover all possible options of success and failure in your tests and use positive and negative tests.

#### Week 4 – suggested deadline: end of week 4

- Change the `TaskManager` class to be a subclass of the collection you are using to store the Tasks in. For example, if you have been using an `ArrayList` in your `TaskManager` to store the Tasks in, you should now make sure that your `TaskManager` extends `ArrayList`. Subsequently, change your implementation of `TaskManager` to not use an `ArrayList` anymore but now be the `ArrayList`. Make sure that all your tests still pass.
- Implement a hierarchy of tasks:
  - `PersonalTask` which should be a subclass of `Task`
  - `WorkTask` which should be a subclass of `Task`
  - `TeamWorkTask` which should be a subclass of `WorkTask`. This task should have an additional field for the `teamName`. Make sure to create all the required constructors.
- Each of the new tasks should contain a method that generates a textual representation of it which states the type of task (i.e. personal, work, or teamwork) as well as the information provided (i.e. ID, name, description, and `teamName`).
- Adjust the user interface to allow the user to create all different types of tasks.
- Adjust your JUnit test to test all different types of tasks.

#### Week 5 – final deadline: beginning of week 7 (check on Canvas)

- Make the `Task` class abstract. Make sure to remove the option to create a `Task` from the user interface. Only `Personal`, `Work`, and `TeamWork` tasks should now be created.
- Implement a `Remindable` interface which contains a `remindUser` method.
- Add the `Remindable` interface to the `WorkTask` and `TeamWorkTask` and implement a simple `remindUser` method that prints a reminder message.
- Implement a way to store all the current tasks on the hard disk when the program exits.
- Implement a way to read a file with tasks from the hard disk when the program is started.

- After the file has been read, print a list of all the current tasks and print all the reminder messages from all the tasks that implement the `Remindable` interface.
- Add a JUnit test to make sure that your File I/O operations work correctly.

## Submission guidelines

### GitLab submissions

You will need to make regular commits to GitLab! You do not have to commit something every week but there has to be a commit for each weekly task. Hence, you need to make a total of 5 commits for this project. Each commit needs to show incremental progress towards the final goal and needs to include a commit message that states what you have achieved since the last commit. There needs to be enough time between each commit to sensibly be able to do all the required work. If you make 5 commits within a single day to finish the whole program, it is very unlikely that you have done the work yourself and we will have to have a discussion with you. **If you do not make incremental commits to GitLab, you will receive 0 marks for this assignment.**

### Canvas submission

Download your final GitLab project as a zip file and commit it to Canvas before the deadline. This ensures we have a backup of your code and tells us if the submission was on time. We will only use the submission time on Canvas to determine if your submission is on time. Hence, make sure to finish your project early enough so you have time for the final submission.

## Plagiarism

This is individual coursework. That means you can discuss the coursework with your peers, but not share your work or use anybody else's, passing it off as your own. You can find more on the general academic misconduct rules here: <https://www.hw.ac.uk/uk/services/academic-registry/academic-integrity/academic-misconduct.htm>.

You are required to do all the work yourself. It is very easy nowadays to just ask an AI to do the work for you and while that might help you finish the coursework; it will not teach you anything. You are here to learn and not just to bide your time until you get a degree. This assignment is meant to prepare you for the remainder of your studies which will require you to know how to program. Getting enough experience programming is, therefore, more important than anything else in 1<sup>st</sup> year and the only way to gain experience is to program!

## Marking Rubric

| Category                    | A                                                                                                                                                                                            | B                                                                                                                                             | C                                                                                                                                                     | D                                                                                                                                                  | E-F                                                                                                                                                                       |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Functionality (30%)</b>  | <b>(30 – 21)</b><br>All the required functionality has been implemented. Extra care has been taken to handle errors and edge cases.                                                          | <b>(20 – 18)</b><br>All or most required functionality has been implemented but there is no or very little handling of errors and edge cases. | <b>(17 – 15)</b><br>At least half of the required functionality has been implemented. There is an attempt to handle edge cases and errors.            | <b>(14 – 12)</b><br>Some of the functionality has been implemented or decent attempts at implementing have been made.                              | <b>(11 – 0)</b><br>An attempt at implementing the most basic functionality has been made.                                                                                 |
| <b>User Interface (20%)</b> | <b>(20 – 14)</b><br>The user interface is verbose enough to inform the user about how to use the system but not overly complicated. It allows to trigger all the implemented functionality.  | <b>(13 – 12)</b><br>The user interface allows to trigger all implemented functions and has some user guidance.                                | <b>(11 – 10)</b><br>The user interface allows to trigger most of the functionality and might have some user guidance.                                 | <b>(9 – 8)</b><br>The user interface allows to trigger some of the functionality and/or is confusing to use.                                       | <b>(7 – 0)</b><br>An attempt has been made at implementing the user interface, but it does not allow triggering the implemented functions and/or is not self-explanatory. |
| <b>Code quality (30%)</b>   | <b>(30 – 21)</b><br>The code follows all the guidelines about responsibility driven design, coupling, and cohesion from the lectures. The code structure is easy to understand and maintain. | <b>(20 – 18)</b><br>The code structure is easy to understand and follows some of the guidelines from the lectures.                            | <b>(17 – 15)</b><br>The code structure can be understood with a little effort. Attempts have been made at following the guidelines from the lectures. | <b>(14 – 12)</b><br>The code is hard to read and maintain but attempts were made to follow the class relationships outlined in the CW description. | <b>(11 – 0)</b><br>The code is hard to understand. None of the guidelines and required class relationships have been implemented.                                         |
| <b>Tests (10%)</b>          | <b>(10 – 7)</b><br>The tests cover all methods. Positive and negative tests are used. Edge cases are tested.                                                                                 | <b>(6)</b><br>The tests cover all methods, but only positive tests are used, and edge cases are only covered partially.                       | <b>(5)</b><br>The tests cover most of the methods but only positive tests are used. Edge cases are not covered.                                       | <b>(4)</b><br>The tests cover some of the methods.                                                                                                 | <b>(3 – 0)</b><br>An attempt has been made at covering some of the methods with tests, but they don't work correctly.                                                     |
| <b>Comments</b>             | <b>(10 – 7)</b>                                                                                                                                                                              | <b>(6)</b>                                                                                                                                    | <b>(5)</b>                                                                                                                                            | <b>(4)</b>                                                                                                                                         | <b>(3 – 0)</b>                                                                                                                                                            |

|       |                                                                                                                                                                                                                        |                                                                                                                                                                           |                                                                                                                                                     |                                                                                                     |                                                   |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------|
| (10%) | <p>The comments are not overly verbose but explain the functionality well. Information about how coupling and cohesion are addressed is included. All files and methods that are not self-explanatory are covered.</p> | <p>Most of the files and methods are covered. Comments are sufficient to understand what has been implemented. Some comments on coupling and cohesion have been made.</p> | <p>Some of the methods and files have comments. The comments are either overly verbose or do not always explain the functionality sufficiently.</p> | <p>Some comments have been made but are not enough to understand what is going on or are wrong.</p> | <p>No or only very few comments can be found.</p> |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------|